

A deterministic ontology-matching defect in the T2I-CompBench 2D-spatial evaluator

Hesam Asadollahzadeh

20 August 2026

Summary

The released T2I-CompBench 2D-spatial evaluator resolves a prompt’s two object nouns to detector outputs by *exact string equality* against a fixed 722-entry label vocabulary. Many of the vocabulary’s entries are composite strings inherited from the unified UniDet label space (`'cattle, cow'`, `'mouse2, mouse'`, `'table, desk'`), and some ordinary nouns have no entry at all. When either noun fails to match, the score is forced to zero before the image is consulted.

318 of the 1,000 prompts in the official `spatial.txt` cannot receive a nonzero score for any image, from any model. Nine nouns account for all of them. On a labelled pool of $298 \text{ prompts} \times 8 \text{ candidates}$ the condition separates perfectly: all 98 affected prompts score zero on all 8 candidates, with no exceptions.

The sibling `3D_spatial_eval.py`, in the same directory, resolves nouns by *substring* containment instead. That single difference is why the 3D category does not show the defect — and it introduces the opposite one, since `cat` is a substring of `'cattle, cow'`.

We rebuild the evaluator with an ontology-aware matcher, verify that our re-implementation reproduces the released scores exactly on 8,642 images, and then ask what changes. Model rankings do **not** change in any statistically defensible way. What does change is everything to do with using the metric as a *target*: repairing the matcher raises the measured best-of- N gain by 29% (from $+0.2143$ to $+0.2760$), 50 prompts per 298 become selectable that were not before, and selecting against the released metric forfeits 25% of the gain that is actually available.

1 The mechanism

1.1 Parsing

The evaluator takes the prompt from the image filename and extracts two nouns with spaCy. The only lexical normalisation anywhere in the file is a four-word person list.

```
nlp = spacy.load("en_core_web_sm")
doc = nlp(prompt)
obj1 = [token.text for token in doc if token.pos_ == 'NOUN'][0]
obj2 = [token.text for token in doc if token.pos_ == 'NOUN'][-1]

person = ['girl', 'boy', 'man', 'woman']
if obj1 in person: obj1 = "person"
if obj2 in person: obj2 = "person"
```

Listing 1: `UniDet_eval/2D_spatial_eval.py`, simple-structure branch.

1.2 The vocabulary

Detected class indices are mapped to strings through a fixed list loaded at import:

```
obj_label_map = torch.load('dataset/detection_features.pt')['labels'] # 722
                  entries
```

Of those 722 entries, 64 are comma-joined composites. The atomic names they contain are not separately addressable, because matching is done against the full string.

1.3 The gate

```
obj = []
for i in range(len(obj_bounding_box)):
    obj_name = obj_label_map[obj_labels_dict[i]]
    obj.append(obj_name)
...
if obj1 in obj and obj2 in obj:
    obj1_pos = obj.index(obj1)
    obj2_pos = obj.index(obj2)
    score = 0.25*instance_score[obj1_pos].item() + 0.25*instance_score[obj2_pos
        ].item()
    score += determine_position(locality, box1, box2) / 2
elif obj1 in obj: score = 0.25*instance_score[obj1_pos].item()
elif obj2 in obj: score = 0.25*instance_score[obj2_pos].item()
else: score = 0
if (score < 0.5): score = 0
```

Listing 2: The defect. `obj` is a *list* of label strings, so `in` is list membership, i.e. exact string equality — not substring containment.

Two consequences follow directly from this block. First, a failed match on *either* noun caps the score at 0.25, which the final threshold then sets to zero: finding one object is worth exactly as much as finding nothing. Second, the score is a joint gate over detection and relation,

$$s_{\text{raw}} = 0.25(c_A + c_B) + 0.5r(A, B), \quad s = \begin{cases} s_{\text{raw}} & s_{\text{raw}} \geq 0.5 \\ 0 & \text{otherwise,} \end{cases}$$

so with typical confidences $c \approx 0.9$ a correct pair of detections and a wrong relation gives $0.45 < 0.5$ and is also zeroed. Detection failure, relation failure and *matching* failure are all reported as the same number.

1.4 Where the names disagree

Table 1: Prompt nouns against the released vocabulary. The right-hand column is what the exact-match gate does.

prompt noun	entry in <code>detection_features.pt</code>	matches?
cow	'cattle, cow'	no
sofa	'sofa bed', 'couch'	no
bag	'handbag'	no
mouse	'mouse2, mouse', 'mouse1'	no
table	'table, desk'	no
wallet	<i>absent</i>	no
giraffe, train, dog	identical string	yes

2 How much of the benchmark this removes

2.1 Repairability

Not every affected prompt can be recovered. Splitting composite labels on commas recovers 115; a five-entry rename map (sofa→couch, bag→handbag, phone→cell phone, computer→laptop, painting→picture) recovers a further 147. The remaining 56 all involve `wallet`, which has no class in the vocabulary under any name; those prompts are not repairable and should be reported as *unsupported* rather than scored zero.

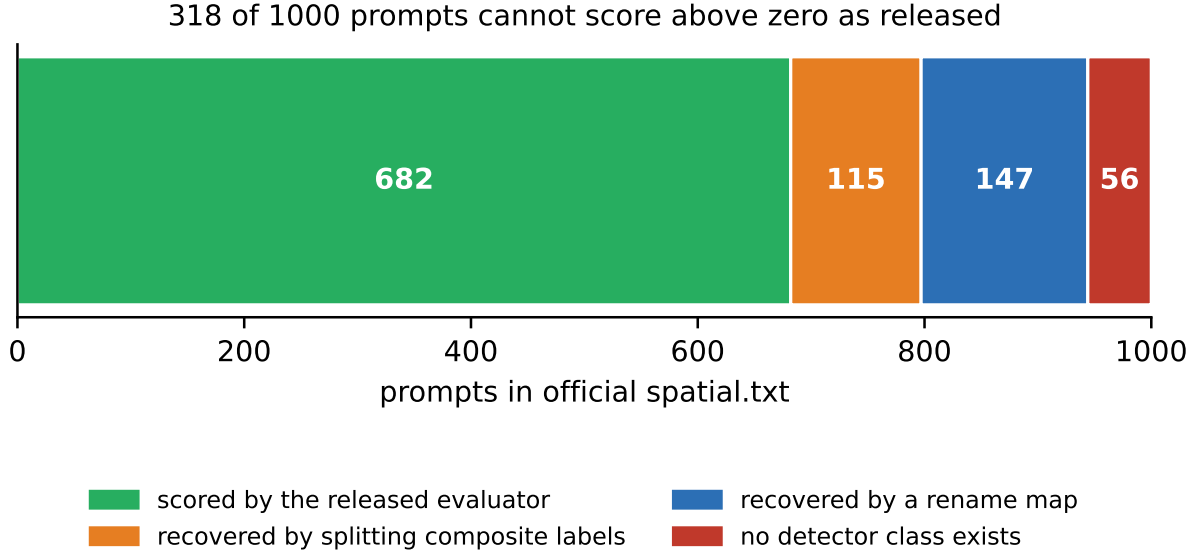


Figure 1: The official 1,000-prompt `spatial.txt`, partitioned by why each prompt does or does not score. Only the leftmost segment is reachable by the released evaluator.

Table 2: Prompts unscorable by construction, official prompt sets. “Unscorable” means the score is 0 for every possible image. The 3D row is shown under both rules; the evaluator that ships for that category uses the substring rule, so its operative figure is 7.4%.

official set	<i>n</i>	exact rule (<code>obj1 in obj</code>)	substring rule (<code>obj1 in obj[i]</code>)
<code>spatial.txt</code>	1000	318 (31.8%)	83 (8.3%)
<code>3d_spatial.txt</code>	1000	331 (33.1%)	74 (7.4%)

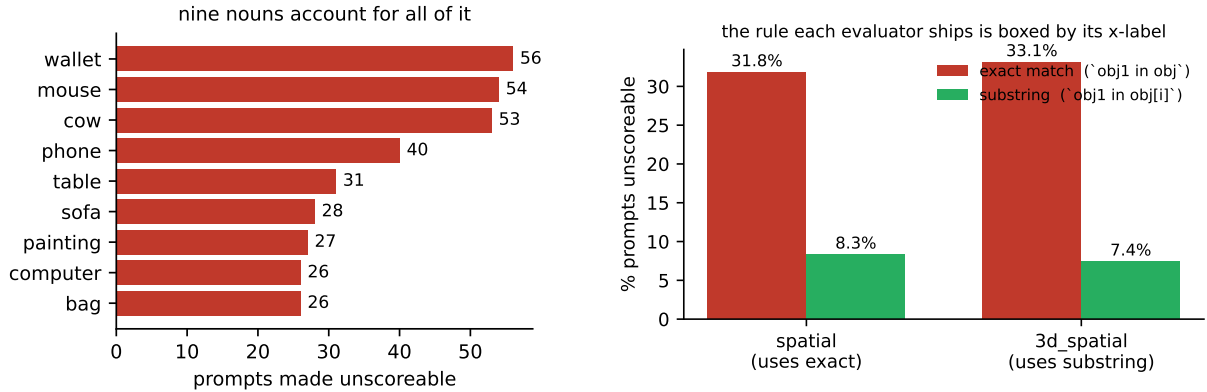


Figure 2: Left: the nine nouns responsible, over the official 1,000 spatial prompts. Right: the same two prompt sets under both matching rules. The two evaluators differ by one character in the matching loop, and that character is the whole difference between the categories.

Table 3: Decomposition of `spatial.txt` by what the prompt needs in order to be scoreable.

status	prompts	share
scored by the released evaluator	682	68.2%
recoverable by splitting composite labels	115	11.5%
recoverable by a rename map	147	14.7%
no detector class exists (<code>wallet</code>)	56	5.6%

3 Verification on a labelled pool

We hold official labels for 298 spatial prompts $\times$ 8 independently seeded candidates from SD3.5-Medium (2,384 images). Predicting “unscorable” from the prompt text alone, before looking at any image:

Table 4: Predicted matchability against measured scores. The separation is exact.

group	prompts	all 8 candidates zero	rate	mean score
both nouns match the vocabulary	200	38	19.0%	0.4281
at least one noun does not	98	98	100.0%	0.0000

Table 5: All-zero rate by relation type. It is roughly flat, which rules out the relation parser as the cause; symmetric relations in fact score highest.

relation type	prompts	all-zero	rate	mean score
top / bottom	107	57	53.3%	0.2139
left / right	106	43	40.6%	0.2765
symmetric (near, next to)	85	36	42.4%	0.3932

3.1 A second layer: the metric is near-binary even where it works

68.2% of the 2,384 candidate images score exactly zero, and 136 of 298 prompts (45.6%) are eight-way ties carrying no candidate-level information at all. Decomposing within-prompt variance into the zero/nonzero split versus graded variation among nonzero scores gives $\eta^2 = 0.9704$ — and the figure is *identical* when restricted to the 200 matchable prompts, because the unscorable prompts contribute no variance either way. So repairing the matcher does not address this second layer: even where the evaluator works, essentially all of its within-prompt signal is whether the joint gate was crossed, not how well the relation was realised.

Official 2D-spatial score under each image. Rows 1-3 are correct images that cannot score above zero; row 4 is a control where both nouns match the detector vocabulary.

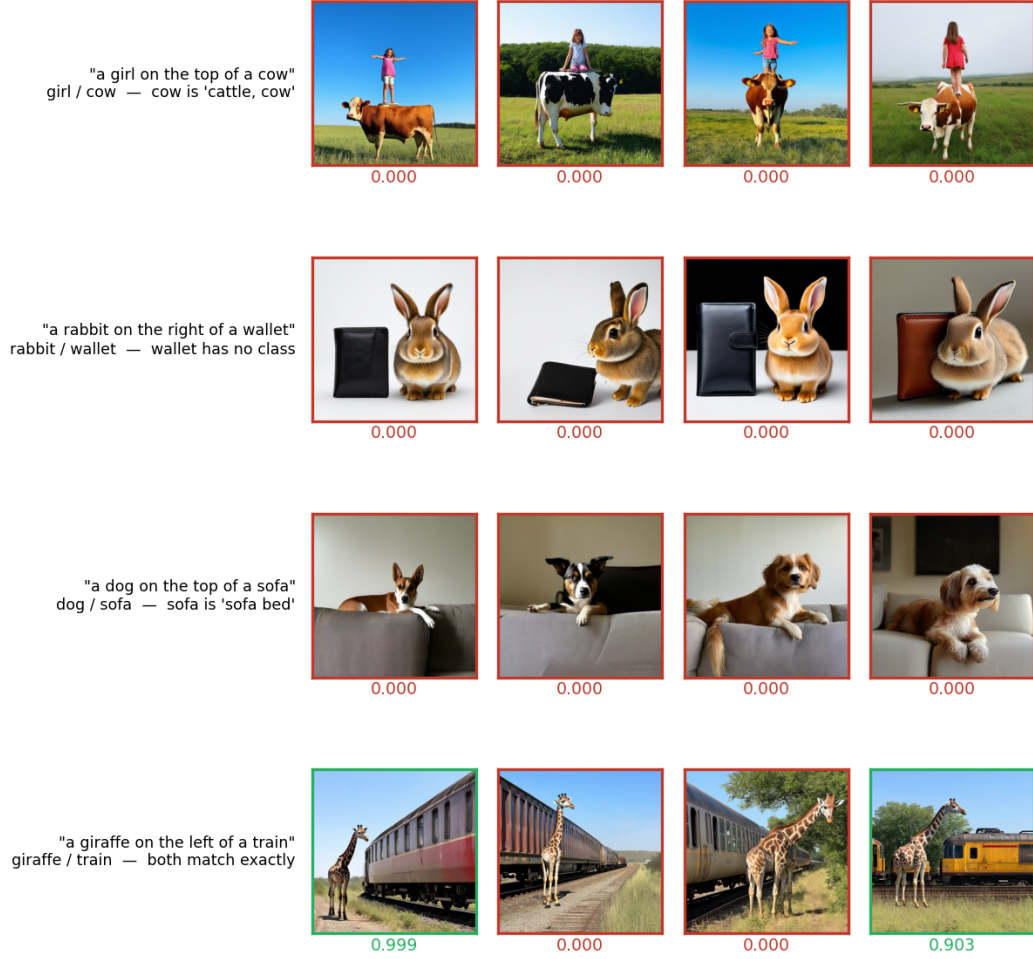
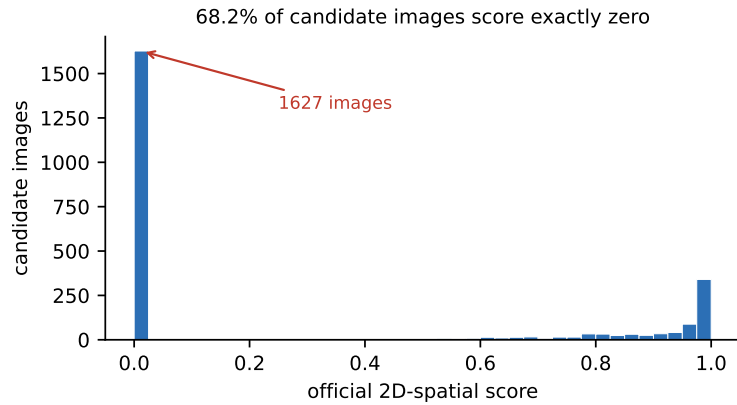


Figure 3: Four candidates for each of four prompts. In rows 1-3 the requested relation is satisfied in every image and every score is exactly zero, because **cow**, **wallet** and **sofa** do not match the vocabulary. Row 4 is a control whose nouns do match: there the evaluator is correct on all four, scoring the two images with the giraffe left of the train and zeroing the two without.



4 The sibling evaluator fails the opposite way

3D_spatial_eval.py uses `if obj1 in obj[i]`, a substring test. That accepts `cow` for `'cattle, cow'`, which is the intended behaviour, but it also accepts:

Table 6: Substring false-positive exposure on the official 3d_spatial.txt. 873 of 1,000 prompts (87.3%) contain a noun that substring-matches an unrelated label. Note this is *exposure*, not an error rate: a spurious match only affects a score if the detector actually fires on that class.

prompt noun	unrelated label it silently matches	prompts exposed
person	'personal care'	211
chair	'wheelchair'	54
book	'bookcase'	48
balloon	'hotair balloon'	44
bee	'frisbee'	44
bag	'cabbage'	41

Neither rule is correct. Splitting composites into atoms and comparing whole atoms accepts `cow` for `'cattle, cow'` while rejecting `cat`.

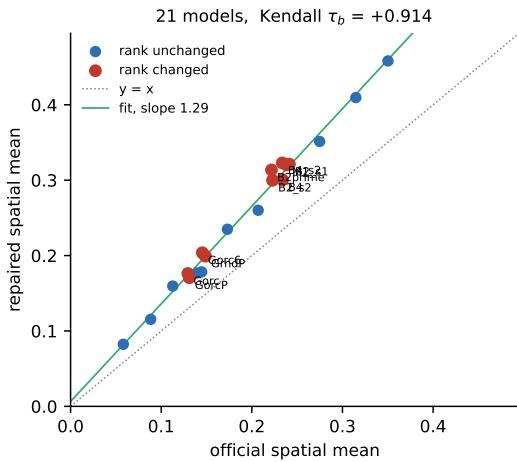
5 A repaired evaluator, and what changes

5.1 Design and validation

The released script discards the detector’s output and keeps one scalar per image, which is why the defect cannot be diagnosed from cached artifacts. We re-ran UniDet over every evaluation for which images survive, kept the raw detections, and scored them offline under three matching rules holding everything else — weighting, `determine_position`, `threshold`, and the first-match `.index()` semantics — identical.

Validation. Our official column reproduces each directory’s own cached `vqa_result.json` on **29/29 directories, 298/298 images each, 8,642 images, zero mismatches**. Any difference in the other columns is therefore attributable to the matching rule alone.

5.2 Model rankings do not meaningfully change



Kendall τ_b , official vs repaired	+0.914
prompt-bootstrap 95% CI	[+0.838, +0.962]
models whose rank moves	10 / 21
top-ranked model	unchanged
pairwise comparisons reversing sign	9 / 210
significant in both directions	0 / 210
per-model repair multiplier	1.233–1.416

Figure 4: Every ranking reversal is between models that were statistically tied to begin with. The repair is not a uniform rescale — the multiplier varies by 0.18 across models, and a pure rescale could not reorder anything — but the variation is not large enough to overturn a defensible comparison.

This is the honest negative result of the audit: **published spatial leaderboards survive**. What does not survive is any use of the metric as an optimisation target.

5.3 Selection changes substantially

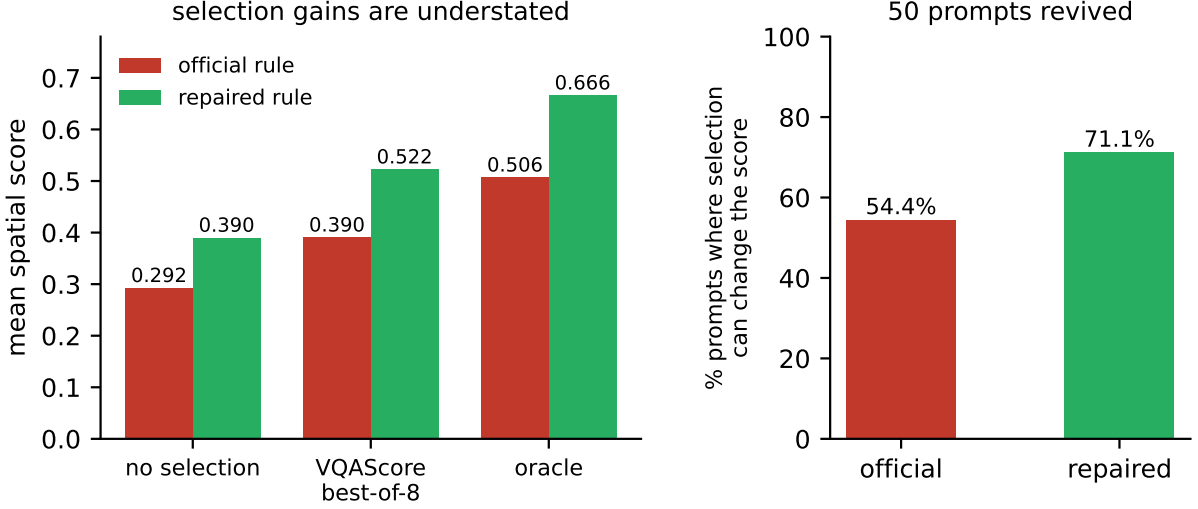


Table 7: Best-of-8 selection over 298 spatial prompts, one model, eight seeds.

	official rule	repaired rule
single candidate	0.2921	0.3896
VQAScore best-of-8	0.3901	0.5221
oracle best-of-8	0.5064	0.6656
oracle gain	+0.2143	+0.2760
prompts where selection can change the score	54.4%	71.1%
prompts revived from dead	50	
prompts where the oracle’s pick changes	48 / 298 (16.1%)	

5.4 The strongest consequence: the metric mis-optimises

Selecting the candidate that maximises the *released* metric, then measuring that candidate under the repaired one, yields +0.2061 against the +0.2760 actually available:

selection policy	realised gain, measured under the repaired rule
argmax of the released metric	+0.2061
argmax of the repaired metric	+0.2760
forfeited	25.3%

Any procedure that uses this metric as a signal — best-of- N selection, rejection sampling, GORS-style fine-tuning on high-scoring generations, or a reward model fit to it — had exactly zero gradient on a third of its prompts, and we can now put a number on what that costs.

5.5 What is unaffected

Prompt-blind selectors built from DINOv2 features do not benefit. The best of them, `quadrant_pc_std`, realises +0.0262 [−0.0185, +0.0695] under the official rule and +0.0380 [−0.0126, +0.0844] under the repaired one: not significant under either. The repair neither rescues nor refutes them.

6 Limitations

1. **Rankings survive.** We cannot claim published model comparisons were wrong. The claim is about absolute scores, effect sizes, and the metric’s use as a target.
2. **One generator.** The labelled pool is SD3.5-Medium. The 318/1,000 figure is a property of the prompts and the evaluator and holds regardless, but the pool statistics are single-model.
3. **The rename map is a judgement.** Five entries, each a rename rather than a claim that two categories are close enough. Results under the comma-split-only rule are also reported.
4. **3D exposure is not an error rate.** Converting 87.3% exposure into a measured false-positive rate requires re-running detection on the 3D pool, which we have not done.
5. **No human validation.** We show the released metric disagrees with the image; we have not run a human study establishing what the corrected metric should say.

7 Reproduction

file	role
phaseX/unidet2d_dump.py	re-runs UniDet, keeps the detections the evaluator discards
phaseX/score_from_dets.py	offline scorer, three matching rules, validates vs cached scores
phaseX/vocab_gate_spatial.json	per-prompt matchability, precomputed
phaseX/make_report_assets.py	recomputes every number and figure in this document
ablations/phaseX_redetect_2d.lsf	the campaign (job 115448)

Every number in this document is read from `reports/figs_compbench/stats.json`, which is regenerated from the artifacts on disk rather than transcribed.